

/*

PART 1: BASICS, VARIABLES, TYPES, COERCION, OPERATORS, CONTROL FLOW

Goal:

- Understand what JavaScript is
- Understand execution environment basics
- Learn variables, scope, hoisting, TDZ
- Master data types, coercion, operators
- Learn how condition-based flow works

Interview rule:

When explaining any concept:

- 1) What is it?
- 2) Why does it exist?
- 3) Why do we use it?
- 4) Example
- 5) Limitation / edge cases
- 6) Best practice

*/

// =====

// [01] WHAT IS JAVASCRIPT

// =====

/*

JavaScript is a high-level, dynamically typed, prototype-based,
multi-paradigm programming language.

Breakdown:

- High-level:

Easier for humans to read/write.

You do not manage memory manually like C/C++.

- Dynamically typed:

Variable types are decided at runtime.

Same variable can hold a number now, string later.

- Prototype-based:

Objects inherit from other objects via prototype chain.

- Multi-paradigm:

Supports procedural, object-oriented, and functional styles.

Main use:

- Browser interactivity

- Server-side development (Node.js)

- Mobile apps

- Desktop apps

- Tooling

- Automation

- Edge/serverless functions

*/

// JavaScript vs ECMAScript

/*

ECMAScript = official language specification (standard/rules)

JavaScript = implementation of that specification

Interview answer:

- ECMAScript defines what the language should do.

- JavaScript is a real-world implementation used in engines like V8.

*/

// Engines

/*

JavaScript engines execute JS code:

- V8 -> Chrome, Node.js
- SpiderMonkey -> Firefox
- JavaScriptCore -> Safari

*/

// Environments

/*

Engine != Environment

Engine:

- Executes JavaScript

Environment:

- Gives JS extra APIs

Examples:

Browser environment:

- window
- document
- localStorage
- fetch

Node.js environment:

- process
- Buffer
- fs
- path
- http

*/

// Single-threaded nature

/*

JavaScript executes on a single main call stack.

That means one operation at a time in normal synchronous execution.

Then how does async work?

- Host environment provides APIs (timers, network, DOM events)
- Event loop schedules callbacks later
- This gives concurrency-like behavior

Important:

- Language execution is single-threaded
- Environment may use background threads internally

*/

// Interpreted vs JIT compiled

/*

Historically JS was called interpreted.

Modern engines use:

- parsing
- interpretation
- Just-In-Time (JIT) compilation
- optimization

Interview-friendly answer:

JavaScript is executed by engines that often combine interpretation and JIT compilation for performance.

*/

// Strict mode

"use strict";

/*

Strict mode:

- catches silent mistakes
- prevents accidental globals
- changes some `this` behavior
- disallows some unsafe patterns

*/

// Case sensitivity

```
const languageName = "JavaScript";
```

```
const LanguageNAME = "Different variable";
```

```
// console.log(languageName); // JavaScript
```

```
// console.log(LanguageNAME); // Different variable
```

// Statements vs Expressions

```
let valueStatement = 10; // statement
```

```
let expressionResult = 2 + 3; // expression produces value 5
```

// Comments

```
// Single-line comment
```

```
/* Multi-line comment */
```

// Semicolons and ASI

/*

ASI = Automatic Semicolon Insertion

JavaScript may insert semicolons automatically.

But relying on ASI can sometimes cause bugs.

*/

```
function badReturnObject() {
```

```
  return
```

```
  {
```

```
    ok: false
```

```
};  
}
```

```
// console.log(badReturnObject());
```

```
// Output: undefined
```

```
// Why? ASI inserted semicolon after return
```

```
function goodReturnObject() {  
  return {  
    ok: true  
  };  
}
```

```
// console.log(goodReturnObject());
```

```
// Output: { ok: true }
```

```
// =====  
// [02] VARIABLES AND DECLARATIONS  
// =====
```

```
/*
```

Three main declaration keywords:

- var
 - let
 - const
- ```
*/
```

```
// var
```

```
/*
```

var:

- function-scoped
- can be redeclared
- can be reassigned

- hoisted and initialized as undefined

Why it exists:

- older JavaScript only had var

Limitation:

- scope leakage
- accidental redeclaration
- confusing hoisting behavior

Modern advice:

- avoid in new code

\*/

```
var oldVar = "old";
```

```
var oldVar = "redeclared"; // allowed
```

```
// let
```

```
/*
```

let:

- block-scoped
- can be reassigned
- cannot be redeclared in same scope
- hoisted but in TDZ before declaration

\*/

```
let age = 25;
```

```
age = 26; // allowed
```

```
// const
```

```
/*
```

const:

- block-scoped

- cannot be reassigned
- must be initialized at declaration

Important:

const does NOT make object contents immutable.

It only prevents rebinding the variable.

\*/

```
const PI = 3.14159;
```

```
// PI = 3.14; // TypeError
```

```
const user = { name: "Dinesh" };
```

```
user.name = "Kumar"; // allowed because object itself is mutable
```

```
// console.log(user.name); // Kumar
```

```
// Variable naming rules
```

/\*

Allowed:

- letters
- digits (not at beginning)
- \_ underscore
- \$ dollar

Not allowed:

- starting with number
- reserved keywords

\*/

```
let userName = "valid";
```

```
let _private = "valid";
```

```
let $dollar = "valid";
```

```
// let 1name = "invalid"; // SyntaxError
```



```
// let class = "invalid"; // SyntaxError
```

```
// Scope
```

```
/*
```

Types of scope:

- Global scope
- Function scope
- Block scope
- Module scope

```
*/
```

```
var globalVar = "global";
```

```
function scopeDemo() {
```

```
 var functionScoped = "only inside function";
```

```
 if (true) {
```

```
 let blockScoped = "inside block";
```

```
 const alsoBlockScoped = "inside block";
```

```
 var stillFunctionScoped = "var ignores block";
```

```
 }
```

```
 // console.log(functionScoped); // works
```

```
 // console.log(stillFunctionScoped); // works
```

```
 // console.log(blockScoped); // ReferenceError
```

```
}
```

```
scopeDemo();
```

```
// Hoisting
```

```
/*
```

Hoisting means declarations are processed before execution.

var:

- hoisted
- initialized to undefined

let/const:

- hoisted
- not initialized
- accessing before declaration causes TDZ error

\*/

```
// console.log(testVar); // undefined
```

```
var testVar = "hello";
```

```
// console.log(testLet); // ReferenceError
```

```
let testLet = "world";
```

```
// TDZ (Temporal Dead Zone)
```

/\*

TDZ = time from entering scope until declaration line is reached

Applies to let/const

\*/

```
{
```

```
 // console.log(myTDZ); // ReferenceError
```

```
 let myTDZ = "now initialized";
```

```
}
```

```
// Best practice
```

/\*

Use:

- const by default
- let if reassignment needed
- avoid var

\*/

```
// =====
```

```
// [03] DATA TYPES
```

```
// =====
```

```
/*
```

Primitive types:

- 1) string
- 2) number
- 3) bigint
- 4) boolean
- 5) undefined
- 6) symbol
- 7) null

Non-primitive:

- object

```
*/
```

```
// Primitive examples
```

```
let fullName = "Dinesh"; // string
```

```
let marks = 98; // number
```

```
let huge = 9007199254740993n; // bigint
```

```
let isActive = true; // boolean
```

```
let notAssigned; // undefined
```

```
let uniqueKey = Symbol("id"); // symbol
```

```
let emptyValue = null; // null
```

```
// Object examples
```

```
let person = { name: "Dinesh" };
```

```
let arr = [1, 2, 3];
```

```
let fn = function () {};
```

```
let dt = new Date();
```

```
// typeof
// console.log(typeof fullName); // string
// console.log(typeof marks); // number
// console.log(typeof huge); // bigint
// console.log(typeof isActive); // boolean
// console.log(typeof notAssigned); // undefined
// console.log(typeof uniqueKey); // symbol
// console.log(typeof emptyValue); // object <-- historical bug/quirk
// console.log(typeof person); // object
// console.log(typeof fn); // function
```

```
// Primitive vs reference
```

```
/*
```

```
Primitive copy -> by value
```

```
Object copy -> by reference
```

```
*/
```

```
let a = 10;
```

```
let b = a;
```

```
b = 20;
```

```
// console.log(a); // 10
```

```
// console.log(b); // 20
```

```
let obj1 = { score: 100 };
```

```
let obj2 = obj1;
```

```
obj2.score = 999;
```

```
// console.log(obj1.score); // 999
```

```
// console.log(obj2.score); // 999
```

```
// Wrapper objects
```

/\*

Primitive wrappers exist:

- String
- Number
- Boolean

Avoid:

new String(...)

new Number(...)

new Boolean(...)

Because they create objects and can cause confusion

\*/

let primitiveStr = "hello";

let objectStr = new String("hello");

// console.log(typeof primitiveStr); // string

// console.log(typeof objectStr); // object

// console.log(primitiveStr === objectStr); // false

// =====

// [04] TYPE CONVERSION AND COERCION

// =====

/\*

Why coercion exists:

- flexibility
- convenience

Problem:

- surprising behavior

Best practice:

- prefer explicit conversion
  - prefer === over ==
- \*/

// Explicit conversion

```
// console.log(String(123)); // "123"
// console.log(Number("123")); // 123
// console.log(Boolean("hello")); // true
// console.log(parseInt("100px", 10)); // 100
// console.log(parseFloat("3.14kg")); // 3.14
```

// Truthy and falsy

/\*

Falsy values:

- false
- 0
- -0
- 0n
- ""
- null
- undefined
- NaN

Everything else is generally truthy.

\*/

```
if ("hello") {
 // console.log("truthy");
}
```

```
if (0) {
 // will not run
}
```

```
}
```

```
// Implicit coercion
```

```
// console.log("5" + 1); // "51" (string concatenation)
```

```
// console.log("5" - 1); // 4 (numeric coercion)
```

```
// console.log(true + 1); // 2
```

```
// console.log(false + 1); // 1
```

```
// Loose vs strict equality
```

```
// console.log(5 == "5"); // true
```

```
// console.log(5 === "5"); // false
```

```
// console.log(null == undefined); // true
```

```
// console.log(null === undefined); // false
```

```
/*
```

Best practice:

Use === and !== almost always.

```
*/
```

```
// NaN
```

```
/*
```

NaN means Not-a-Number.

Important:

NaN !== NaN

```
*/
```

```
// console.log(Number("abc")); // NaN
```

```
// console.log(NaN === NaN); // false
```

```
// console.log(Number.isNaN(NaN)); // true
```

```
// Infinity
```

```
// console.log(1 / 0); // Infinity
```

```
// console.log(-1 / 0); // -Infinity
```

```
// Object.is()
```

```
/*
```

Object.is is similar to === but:

- Object.is(NaN, NaN) => true

- Object.is(+0, -0) => false

```
*/
```

```
// console.log(Object.is(NaN, NaN)); // true
```

```
// console.log(Object.is(+0, -0)); // false
```

```
// console.log(+0 === -0); // true
```

```
// =====
```

```
// [05] OPERATORS
```

```
// =====
```

```
// Arithmetic
```

```
let x = 10;
```

```
let y = 3;
```

```
// console.log(x + y); // 13
```

```
// console.log(x - y); // 7
```

```
// console.log(x * y); // 30
```

```
// console.log(x / y); // 3.333...
```

```
// console.log(x % y); // 1
```

```
// console.log(x ** y); // 1000
```

```
let count = 1;
```

```
count++;
```

```
++count;
```

```
// console.log(count); // 3
```

```
// Assignment
```



```
let total = 10;

total += 5;

total -= 2;

total *= 2;

total /= 2;

// console.log(total); // 13

// Comparison

// console.log(5 > 3); // true
// console.log(5 < 3); // false
// console.log(5 >= 5); // true
// console.log(5 <= 4); // false
// console.log(5 === "5");// true
// console.log(5 == "5");// false

// Logical

// console.log(true && false); // false
// console.log(true || false); // true
// console.log(!true); // false

// Short-circuiting

// console.log(false && "run"); // false
// console.log(true || "run"); // true

// Ternary operator

let ageLimit = 20;

let voting = ageLimit >= 18 ? "Allowed" : "Not allowed";

// console.log(voting); // Allowed

// typeof

// console.log(typeof 123); // number

// delete
```

```
let book = { title: "JS Guide", price: 500 };
```

```
delete book.price;
```

```
// console.log(book); // { title: 'JS Guide' }
```

```
// in
```

```
// console.log("title" in book); // true
```

```
// instanceof
```

```
let items = [];
```

```
// console.log(items instanceof Array); // true
```

```
// Nullish coalescing ??
```

```
/*
```

```
?? returns right side only if left is null or undefined.
```

```
Better than || when 0, false, "" are valid values.
```

```
*/
```

```
let quantity = 0;
```

```
// console.log(quantity || 10); // 10 (problem if 0 is valid)
```

```
// console.log(quantity ?? 10); // 0 (correct)
```

```
// Optional chaining ?.
```

```
let company = {
```

```
 department: {
```

```
 name: "Engineering"
```

```
 }
```

```
};
```

```
// console.log(company.department?.name); // Engineering
```

```
// console.log(company.hr?.name); // undefined
```

```
// Spread ...
```

```
let arr1 = [1, 2];

let arr2 = [...arr1, 3, 4];

// console.log(arr2); // [1,2,3,4]

let objA = { a: 1 };

let objB = { ...objA, b: 2 };

// console.log(objB); // { a:1, b:2 }

// Rest ...

function sumAll(...nums) {

 return nums.reduce((sum, n) => sum + n, 0);

}

// console.log(sumAll(1,2,3,4)); // 10
```

// Bitwise operators

/\*

Used for low-level/binary operations:

& | ^ ~ << >> >>>

\*/

```
// console.log(5 & 1); // 1
```

```
// console.log(5 | 1); // 5
```

```
// console.log(5 ^ 1); // 4
```

// Comma operator

```
let commaValue = (1, 2, 3);
```

```
// console.log(commaValue); // 3
```

// =====

// [06] CONTROL FLOW

// =====

// if / else if / else

```
let score = 78;
```

```
if (score >= 90) {
 // console.log("Grade A");
} else if (score >= 75) {
 // console.log("Grade B");
} else {
 // console.log("Need improvement");
}
```

```
// switch
let fruit = "apple";
```

```
switch (fruit) {
 case "mango":
 // console.log("Summer fruit");
 break;
 case "apple":
 // console.log("Common fruit");
 break;
 default:
 // console.log("Unknown fruit");
}
```

```
// break / continue
for (let i = 0; i < 5; i++) {
 if (i === 2) continue;
 if (i === 4) break;
 // console.log(i); // 0,1,3
}
```

```
// return
function divide(a, b) {
 if (b === 0) return "Cannot divide by zero";
```

```
return a / b;

}

// console.log(divide(10, 2)); // 5
// console.log(divide(10, 0)); // Cannot divide by zero
```

```
/*
```

Best practice:

- use early return to reduce nesting
- keep conditions clear and readable
- extract complex conditions into named functions

```
*/
```

```
`
```

-----Part 2-----

```
/*
```

---

## PART 2: LOOPS, STRINGS, NUMBERS, BOOLEANS, ARRAYS, OBJECTS

---

```
*/
```

```
// =====
```

```
// [07] LOOPS AND ITERATION
```

```
// =====
```

```
/*
```

Loops let us repeat work.

Main types:

- for
- while
- do...while
- for...in
- for...of

```
*/
```

```
for (let i = 0; i < 3; i++) {
 // console.log("for:", i);
}
```

```
let w = 0;
while (w < 2) {
 // console.log("while:", w);
 w++;
}
```

```
let d = 0;
do {
 // console.log("do while runs at least once:", d);
 d++;
} while (d < 1);
```

```
// for...in -> object keys
let employee = { id: 1, name: "Sam" };
for (let key in employee) {
 // console.log(key, employee[key]);
}
```

```
// for...of -> iterable values
for (let value of [10, 20, 30]) {
 // console.log(value);
}
```

```
/*
```

Difference:

- for...in -> keys/properties
- for...of -> values from iterable

```
*/
```

```
// labeled statements
```

```
outerLoop:
```

```
for (let i = 0; i < 3; i++) {
 for (let j = 0; j < 3; j++) {
 if (i === 1 && j === 1) break outerLoop;
 }
}
```

```
/*
```

Rarely used in production because labels reduce readability.

```
*/
```

```
// =====
```

```
// [08] STRINGS
```

```
// =====
```

```
/*
```

Strings:

- sequence of UTF-16 code units
- immutable

Why immutable?

- safety
- implementation simplicity
- reliable behavior

```
*/
```

```
let firstName = "Dinesh";
```

```
let lastName = 'Kumar';
```

```
let full = `${firstName} ${lastName}`;
```

```
// console.log(full); // Dinesh Kumar
```

```
// Multiline strings
```

```
let multi = `Line 1
```

```
Line 2
```

```
Line 3`;
```

```
// console.log(multi);
```

```
// Immutability
```

```
let original = "cat";
```

```
let upper = original.toUpperCase();
```

```
// console.log(original); // cat
```

```
// console.log(upper); // CAT
```

```
// Common methods
```

```
let text = " JavaScript Interview ";
```

```
// console.log(text.length); // includes spaces
```

```
// console.log(text.trim()); // "JavaScript Interview"
```

```
// console.log(text.toUpperCase()); // uppercase
```

```
// console.log(text.toLowerCase()); // lowercase
```

```
// console.log(text.slice(2, 12)); // substring by indexes
```

```
// console.log(text.substring(2, 12)); // similar
```

```
// console.log(text.replace("Interview", "Guide")); // replaces first match
```

```
// console.log("a-b-c".replaceAll("-", ":")); // a:b:c
```

```
// console.log("a,b,c".split(",")); // ['a', 'b', 'c']
```

```
// console.log(text.includes("Script")); // true
```

```
// console.log(text.startsWith(" J")); // true
```

```
// console.log(text.endsWith(" ")); // true
```

```
// console.log(text.indexOf("Script")); // index
```

```
// console.log("5".padStart(3, "0")); // 005
```



```
// console.log("5".padEnd(3, "0")); // 500
// console.log("ha".repeat(3)); // hahaha
```

```
// Unicode/UTF discussion
```

```
/*
Important interview point:
string length is based on UTF-16 code units, not always visible characters.
*/
```

```
let smile = "😊";
// console.log(smile.length); // 2
// console.log([...smile].length); // 1
```

```
// Escape sequences
```

```
let escaped = "Hello\nWorld\t!!!";
// console.log(escaped);
```

```
// =====
// [09] NUMBERS AND MATH
// =====
```

```
/*
JavaScript number type:
- IEEE-754 double precision floating point
```

```
That means:
- integers and decimals use same number type
- floating precision issues can happen
*/
```

```
// console.log(0.1 + 0.2); // 0.30000000000000004
```

```
/*
```

Why?

Binary floating point cannot represent some decimal fractions exactly.

Solution in finance:

- store money as smallest unit (paise/cents)

- or use decimal libraries

\*/

// Safe integers

// console.log(Number.MAX\_SAFE\_INTEGER); // 9007199254740991

// console.log(Number.MAX\_SAFE\_INTEGER + 1 === Number.MAX\_SAFE\_INTEGER + 2); // true

// BigInt

let veryBig = 9007199254740993n;

// console.log(veryBig + 2n); // 9007199254740995n

/\*

Limitation:

Cannot mix BigInt and Number directly.

\*/

// console.log(1n + 1); // TypeError

// Number methods

// console.log(Number.isNaN(NaN)); // true

// console.log(Number.isInteger(10.0)); // true

// console.log(Number.parseInt("42px", 10)); // 42

// console.log(Number.parseFloat("3.14kg")); // 3.14

// console.log((3.14159).toFixed(2)); // "3.14"

// console.log((3.14159).toPrecision(3)); // "3.14"

// Math

// console.log(Math.round(4.6)); // 5

// console.log(Math.floor(4.9)); // 4

```
// console.log(Math.ceil(4.1)); // 5
// console.log(Math.trunc(4.9)); // 4
// console.log(Math.max(1, 5, 2)); // 5
// console.log(Math.min(1, 5, 2)); // 1
// console.log(Math.pow(2, 3)); // 8
// console.log(Math.sqrt(16)); // 4
// console.log(Math.abs(-10)); // 10
```

```
function getRandomInt(min, max) {
 // inclusive range
 return Math.floor(Math.random() * (max - min + 1)) + min;
}
```

```
// console.log(getRandomInt(1, 6)); // random number between 1 and 6
```

```
// =====
```

```
// [10] BOOLEANS
```

```
// =====
```

```
/*
```

Booleans represent logical truth:

- true

- false

```
*/
```

```
let isLoggedIn = true;
```

```
let hasSubscription = false;
```

```
// console.log(isLoggedIn && hasSubscription); // false
```

```
// console.log(isLoggedIn || hasSubscription); // true
```

```
// console.log(!isLoggedIn); // false
```

```
/*
```

Short-circuit behavior:

&& returns first falsy or last truthy

|| returns first truthy or last falsy

\*/

```
let userName = "";
```

```
let displayName = userName || "Guest";
```

```
// console.log(displayName); // Guest
```

```
// =====
```

```
// [11] ARRAYS
```

```
// =====
```

/\*

Arrays store ordered collections.

Why use arrays?

- maintain order

- indexed access

- easy iteration

- many built-in methods

\*/

```
let numbers = [10, 20, 30];
```

```
// console.log(numbers[0]); // 10
```

```
numbers[1] = 25;
```

```
// console.log(numbers); // [10,25,30]
```

```
// length
```

```
// console.log(numbers.length); // 3
```

```
// Sparse arrays
```

```
let sparse = [];
```

```
sparse[5] = "X";
```

```
// console.log(sparse.length); // 6
```

```
// console.log(sparse);
```

```
/*
```

Sparse arrays are usually avoided because many methods behave unexpectedly.

```
*/
```

```
// Multidimensional arrays
```

```
let matrix = [
```

```
 [1, 2],
```

```
 [3, 4]
```

```
];
```

```
// console.log(matrix[1][0]); // 3
```

```
// Mutating methods
```

```
let fruits = ["apple", "banana"];
```

```
fruits.push("mango"); // add at end
```

```
fruits.pop(); // remove from end
```

```
fruits.shift(); // remove from start
```

```
fruits.unshift("grape");// add at start
```

```
fruits.splice(1, 0, "orange");// insert at index
```

```
fruits.reverse(); // mutate original
```

```
fruits.sort(); // lexical sort
```

```
// console.log(fruits);
```

```
// Sort pitfall
```

```
let nums = [100, 2, 20];
```

```
nums.sort();
```

```
// console.log(nums); // [100, 2, 20] lexical sort, not numeric
```

```
nums.sort((a, b) => a - b);

// console.log(nums); // [2,20,100]

// Non-mutating methods

let arr1 = [1, 2, 3];

let arr2 = [4, 5];

// console.log(arr1.concat(arr2)); // [1,2,3,4,5]
// console.log(arr1.slice(1)); // [2,3]
// console.log(arr1.map(x => x * 2)); // [2,4,6]
// console.log(arr1.filter(x => x > 1)); // [2,3]
// console.log(arr1.reduce((sum, x) => sum + x, 0)); // 6
// console.log(arr1.find(x => x > 1)); // 2
// console.log(arr1.findIndex(x => x === 3)); // 2
// console.log(arr1.some(x => x > 2)); // true
// console.log(arr1.every(x => x > 0)); // true
// console.log(arr1.includes(2)); // true
// console.log(arr1.join("-")); // "1-2-3"
// console.log([1, [2, [3]]].flat(2)); // [1,2,3]
// console.log([1,2].flatMap(x => [x, x*10])); // [1,10,2,20]

// New non-mutating methods

let modern = [3, 1, 2];

// console.log(modern.toSorted((a, b) => a - b)); // [1,2,3]
// console.log(modern.toReversed()); // [2,1,3]
// console.log(modern.toSpliced(1, 1, 99)); // [3,99,2]
// console.log(modern); // original unchanged

// Iteration methods

[1, 2, 3].forEach((value, index) => {
 // console.log(index, value);
});
```

```
// Destructuring
```

```
let [first, second, ...rest] = [10, 20, 30, 40];
```

```
// console.log(first, second, rest); // 10 20 [30,40]
```

```
// Shallow copy vs deep copy
```

```
let nestedA = [{ val: 1 }];
```

```
let nestedB = [...nestedA];
```

```
nestedB[0].val = 999;
```

```
// console.log(nestedA[0].val); // 999 because nested object reference shared
```

```
let deepOriginal = { user: { name: "A" } };
```

```
let deepClone = structuredClone(deepOriginal);
```

```
deepClone.user.name = "B";
```

```
// console.log(deepOriginal.user.name); // A
```

```
// console.log(deepClone.user.name); // B
```

```
// =====
```

```
// [12] OBJECTS
```

```
// =====
```

```
/*
```

Objects store key-value pairs.

Used to model real-world data:

- user

- product

- order

- config

```
*/
```

```
// Creating objects
```

```
let person = {
 name: "Dinesh",
 age: 30,
 greet() {
 return `Hello, ${this.name}`;
 }
};

// console.log(person.name); // Dinesh
// console.log(person["age"]); // 30
// console.log(person.greet()); // Hello, Dinesh
```

```
// Dynamic property
let key = "city";
person[key] = "Coimbatore";
// console.log(person.city); // Coimbatore
```

```
// Computed property names
let dynamicKey = "score";
let result = {
 [dynamicKey]: 95
};
// console.log(result.score); // 95
```

```
// Nested objects
```

```
let company = {
 name: "ABC",
 address: {
 city: "Chennai",
 pin: 600001
 }
};
```



```
// console.log(company.address.city); // Chennai
```

```
// Symbol keys
```

```
let secret = Symbol("secret");
```

```
let securedObj = {
```

```
 [secret]: "hidden"
```

```
};
```

```
// console.log(securedObj[secret]); // hidden
```

```
// Object utility methods
```

```
// console.log(Object.keys(person));
```

```
// console.log(Object.values(person));
```

```
// console.log(Object.entries(person));
```

```
let merged = Object.assign({}, { a: 1 }, { b: 2 });
```

```
// console.log(merged); // { a:1, b:2 }
```

```
// freeze / seal / preventExtensions
```

```
let settings = { theme: "dark" };
```

```
Object.freeze(settings);
```

```
// settings.theme = "light"; // fails
```

```
// console.log(settings.theme); // dark
```

```
let profile = { name: "Sam" };
```

```
Object.seal(profile);
```

```
profile.name = "John"; // allowed
```

```
// delete profile.name; // not allowed
```

```
let config = { mode: "dev" };
```

```
Object.preventExtensions(config);
```

```
// config.newProp = 123; // not allowed
```

```
// hasOwnProperty / Object.hasOwn
```

```
// console.log(person.hasOwnProperty("name")); // true
```

```
// console.log(Object.hasOwn(person, "name")); // true
```

```
// Property descriptors
```

```
let descriptorObj = {};
```

```
Object.defineProperty(descriptorObj, "id", {
 value: 101,
 writable: false,
 enumerable: true,
 configurable: false
});
```

```
// descriptorObj.id = 999; // fails
```

```
// console.log(descriptorObj.id); // 101
```

```
// Getters and setters
```

```
let temperature = {
 _celsius: 0,
 get fahrenheit() {
 return this._celsius * 9 / 5 + 32;
 },
 set celsius(value) {
 if (typeof value !== "number") {
 throw new TypeError("Must be a number");
 }
 this._celsius = value;
 }
};
```

```
temperature.celsius = 25;
```

```
// console.log(temperature.fahrenheit); // 77
```

```
// Object.create
```

```
let animalProto = {
 speak() {
 return `${this.name} makes a sound`;
 }
};

let dog = Object.create(animalProto);
dog.name = "Tommy";

// console.log(dog.speak()); // Tommy makes a sound
```

```
/*
```

Object best practices:

- prefer object literals for simplicity
- use bracket notation for dynamic keys
- use optional chaining for nested access
- beware shallow copies

```
*/
```

-----Part 3-----

```
/*
```

---

PART 3: FUNCTIONS, SCOPE, THIS, PROTOTYPES, CLASSES, ERROR HANDLING

---

```
*/
```

```
// =====
```

```
// [13] FUNCTIONS
```

```
// =====
```

```
/*
```

Functions are first-class citizens:

- can be assigned to variables
- passed as arguments
- returned from functions

```
*/
```

```
function add(a, b) {
```

```
 return a + b;
```

```
}
```

```
// console.log(add(2, 3)); // 5
```

```
// Function expression
```

```
const subtract = function (a, b) {
```

```
 return a - b;
```

```
};
```

```
// console.log(subtract(5, 2)); // 3
```

```
// Named function expression
```

```
const factorialExpr = function factorial(n) {
```

```
 if (n <= 1) return 1;
```

```
 return n * factorial(n - 1);
```

```
};
```

```
// console.log(factorialExpr(5)); // 120
```

```
// Arrow functions
```

```
const multiply = (a, b) => a * b;
```

```
// console.log(multiply(3, 4)); // 12
```

```
/*
```

Arrow functions:

Pros:

- concise
- lexical this

Cons:

- no own this
- no own arguments
- cannot be used as constructors

```
*/
```

```
// Parameters / arguments
```

```
function greet(name = "Guest") {
 return `Hello, ${name}`;
}
```

```
// console.log(greet()); // Hello, Guest
```

```
function sumAll(...nums) {
 return nums.reduce((sum, n) => sum + n, 0);
}
```

```
// console.log(sumAll(1,2,3)); // 6
```

```
// Higher-order function
```

```
function operate(a, b, fn) {
 return fn(a, b);
}
```

```
// console.log(operate(10, 5, add)); // 15
```

```
// Callback
```

```
setTimeout(() => {
 // console.log("Callback called later");
}, 10);
```

```
// Pure functions
```

```
function pureAdd(a, b) {
```

```
 return a + b;
}
```

```
/*
```

Pure function:

- same input -> same output
- no side effects

```
*/
```

```
let externalCounter = 0;

function incrementCounter() {
 externalCounter++; // side effect
}
```

```
// Recursion
```

```
function factorial(n) {
 if (n <= 1) return 1;
 return n * factorial(n - 1);
}

// console.log(factorial(5)); // 120
```

```
// IIFE
```

```
(function () {
 let hidden = "private";
 // console.log(hidden);
})();
```

```
// arguments object
```

```
function showArgs() {
 // console.log(arguments);
}

showArgs(1, 2, 3);
```

// Function hoisting

hoistedFunction();

function hoistedFunction() {

// console.log("Function declarations are hoisted");

}

// Closure

function outer() {

let count = 0;

return function inner() {

count++;

return count;

};

}

const counter = outer();

// console.log(counter()); // 1

// console.log(counter()); // 2

/\*

Why closure exists:

Inner function remembers lexical variables

even after outer function finishes.

Use cases:

- private state

- module/factory patterns

- memoization

- event handlers

\*/

// Scope chain / lexical scope

let globalMessage = "global";

```
function one() {
 let local1 = "one";

 function two() {
 let local2 = "two";
 // console.log(globalMessage, local1, local2);
 }

 two();
}

one();

// call, apply, bind
function introduce(city, country) {
 return `I am ${this.name} from ${city}, ${country}`;
}

const dev = { name: "Dinesh" };

// console.log(introduce.call(dev, "Coimbatore", "India"));
// console.log(introduce.apply(dev, ["Coimbatore", "India"]));

const boundFn = introduce.bind(dev, "Coimbatore");
// console.log(boundFn("India"));

// Currying
function curryAdd(a) {
 return function (b) {
 return a + b;
 };
}
```



```
// console.log(curryAdd(2)(3)); // 5
```

```
// Partial application
```

```
function volume(length, width, height) {
 return length * width * height;
}
```

```
const fixedBase = volume.bind(null, 10, 5);
```

```
// console.log(fixedBase(2)); // 100
```

```
// Function composition
```

```
const double = x => x * 2;
```

```
const square = x => x * x;
```

```
const compose = (f, g) => x => f(g(x));
```

```
const squareThenDouble = compose(double, square);
```

```
// console.log(squareThenDouble(3)); // 18
```

```
// Memoization
```

```
function createMemoizedSquare() {
```

```
 const cache = new Map();
```

```
 return function (n) {
```

```
 if (cache.has(n)) return cache.get(n);
```

```
 const result = n * n;
```

```
 cache.set(n, result);
```

```
 return result;
```

```
 };
```

```
}
```

```
const memoSquare = createMemoizedSquare();
```

```
// console.log(memoSquare(5)); // 25
```

```
// console.log(memoSquare(5)); // cached 25
```

```
// Debounce
```

```
function debounce(fn, delay) {
 let timerId;

 return function (...args) {
 clearTimeout(timerId);
 timerId = setTimeout(() => fn.apply(this, args), delay);
 };
}
```

```
/*
```

Use case:

search box -> wait until user stops typing before API call

```
*/
```

```
// Throttle
```

```
function throttle(fn, interval) {
 let lastTime = 0;

 return function (...args) {
 const now = Date.now();

 if (now - lastTime >= interval) {
 lastTime = now;
 fn.apply(this, args);
 }
 };
}
```

```
/*
```

Use case:

scroll / resize event so callback doesn't run too frequently

\*/

// =====

// [14] SCOPE AND EXECUTION CONTEXT

// =====

/\*

Execution context:

internal engine concept storing info needed to run code

Types:

- global execution context
- function execution context
- eval execution context (rare, avoid eval)

\*/

// Call stack

function first() {

    second();

}

function second() {

    third();

}

function third() {

    // console.log("Reached third()");

}

// first();

/\*

Call stack is LIFO:

first -> second -> third

then unwinds back

```
*/
```

```
// Stack overflow
```

```
function recurseForever() {
```

```
 return recurseForever();
```

```
}
```

```
// recurseForever(); // RangeError: Maximum call stack size exceeded
```

```
// =====
```

```
// [15] THIS KEYWORD
```

```
// =====
```

```
/*
```

`this` depends on HOW a function is called,

not where it was written (except arrow functions).

```
*/
```

```
function regularThis() {
```

```
 return this;
```

```
}
```

```
// In strict mode plain call => undefined
```

```
// console.log(regularThis());
```

```
const userObj = {
```

```
 name: "Alex",
```

```
 show() {
```

```
 return this.name;
```

```
 }
```

```
};
```

```
// console.log(userObj.show()); // Alex
```

```
// Constructor this
```

```
function Person(name) {
 this.name = name;
}

const person = new Person("Sam");

// console.log(person.name); // Sam
```

```
// Event handler this (browser)
```

```
/*
```

In regular event listeners, `this` usually refers to the current element.

Arrow functions do not get their own `this`.

```
*/
```

```
// Arrow lexical this
```

```
const arrowDemo = {
 name: "Outer",
 regular() {
 const arrow = () => this.name;
 return arrow();
 }
};

// console.log(arrowDemo.regular()); // Outer
```

```
/*
```

Best practice:

- use regular functions for methods when you need dynamic this
- use arrows for callbacks where lexical this is useful

```
*/
```

```
// =====
```

```
// [16] PROTOTYPES AND INHERITANCE
```

```
// =====
```

```
/*
```

JavaScript uses prototype-based inheritance.

Why?

- memory-efficient method sharing
- objects delegate property lookup to prototypes

\*/

```
function Animal(name) {
 this.name = name;
}
```

```
Animal.prototype.speak = function () {
 return `${this.name} makes a sound`;
};
```

```
const dog = new Animal("Dog");

// console.log(dog.speak()); // Dog makes a sound

// console.log(dog.hasOwnProperty("name")); // true

// console.log(dog.hasOwnProperty("speak")); // false
```

/\*

prototype property:

- exists on functions used as constructors

\_\_proto\_\_:

- old accessor to internal prototype
- avoid in production; prefer Object.getPrototypeOf

\*/

// Inheritance via prototypes

```
function Dog(name, breed) {
 Animal.call(this, name);
 this.breed = breed;
```

```
}
```

```
Dog.prototype = Object.create(Animal.prototype);
```

```
Dog.prototype.constructor = Dog;
```

```
Dog.prototype.bark = function () {
```

```
 return `${this.name} barks`;
```

```
};
```

```
const buddy = new Dog("Buddy", "Labrador");
```

```
// console.log(buddy.speak()); // Buddy makes a sound
```

```
// console.log(buddy.bark()); // Buddy barks
```

```
// Property shadowing
```

```
Animal.prototype.type = "generic animal";
```

```
buddy.type = "domestic dog";
```

```
// console.log(buddy.type); // domestic dog
```

```
// =====
```

```
// [17] CLASSES
```

```
// =====
```

```
/*
```

```
Classes are syntactic sugar over prototypes.
```

```
They make object-oriented code look cleaner.
```

```
*/
```

```
class Vehicle {
```

```
 static category = "Transport";
```

```
 #secretCode = 12345; // private field
```

```
 constructor(brand) {
```

```
 this.brand = brand;
}

start() {
 return `${this.brand} started`;
}

get secret() {
 return this.#secretCode;
}

set secret(value) {
 if (typeof value !== "number") {
 throw new TypeError("Secret must be a number");
 }
 this.#secretCode = value;
}

static info() {
 return "Vehicles help transportation";
}
}

class Car extends Vehicle {
 constructor(brand, model) {
 super(brand);
 this.model = model;
 }

 start() {
 return `${super.start()} -> model ${this.model}`;
 }
}
```



```
const myCar = new Car("Toyota", "Corolla");

// console.log(myCar.start()); // Toyota started -> model Corolla

// console.log(Car.info()); // Vehicles help transportation

// console.log(Vehicle.category); // Transport

// console.log(myCar.secret); // 12345
```

```
/*
```

Class concepts:

- constructor
- instance methods
- static methods/properties
- extends
- super
- getters/setters
- public fields
- private fields

```
*/
```

```
// Class expressions
```

```
const Box = class {
 constructor(value) {
 this.value = value;
 }
};
```

```
const bx = new Box(10);

// console.log(bx.value); // 10
```

```
// =====
```

```
// [18] ERROR HANDLING
```

```
// =====
```

```
/*
```

Error types:

- SyntaxError
- ReferenceError
- TypeError
- RangeError
- URIError
- EvalError
- generic Error

```
*/
```

```
// try/catch/finally
```

```
try {
 JSON.parse("invalid json");
} catch (error) {
 // console.log(error.name); // SyntaxError
 // console.log(error.message); // parsing message
} finally {
 // console.log("Cleanup if needed");
}
```

```
// throw
```

```
function divideStrict(a, b) {
 if (b === 0) {
 throw new RangeError("Denominator cannot be zero");
 }
 return a / b;
}
```

```
// console.log(divideStrict(10, 2)); // 5
```

```
// console.log(divideStrict(10, 0)); // throws
```

```
// Custom error
```

```

class ValidationError extends Error {
 constructor(message, field) {
 super(message);
 this.name = "ValidationError";
 this.field = field;
 }
}

function validateEmail(email) {
 if (!email.includes("@")) {
 throw new ValidationError("Invalid email format", "email");
 }
 return true;
}

try {
 validateEmail("abc.com");
} catch (err) {
 // console.log(err.name); // ValidationError
 // console.log(err.field); // email
 // console.log(err.message); // Invalid email format
}

```

/\*

Best practices:

- throw meaningful errors
- catch where you can actually recover
- avoid swallowing errors silently
- attach context when useful

\*/

/\*

---

---

## PART 4: DATES, REGEX, JSON, ADVANCED BUILT-INS, DESTRUCTURING, MODULES

---

---

\*/

// =====

// [19] DATES AND TIME

// =====

/\*

Date stores milliseconds since Unix epoch:

Jan 1, 1970 UTC

\*/

let now = new Date();

// console.log(now.toString());

// console.log(now.toISOString());

let timestamp = Date.now();

// console.log(timestamp);

let parsedDate = new Date("2026-06-17T10:00:00Z");

// console.log(parsedDate.getUTCFullYear()); // 2026

// console.log(parsedDate.getMonth()); // 0-based month

let future = new Date(now);

future.setDate(now.getDate() + 7);

// console.log(future.toDateString());

/\*

Date limitations:

- parsing ambiguity

- timezone complexity
- daylight saving issues
- old API design

Best practice:

- store UTC/timestamps
- format at display time

\*/

// =====

// [20] REGULAR EXPRESSIONS

// =====

/\*

Regex = pattern matching language for strings

Use cases:

- validation
- search
- replace
- parsing

\*/

let regex = /cat/;

// console.log(regex.test("my cat")); // true

let regexInsensitive = /hello/i;

// console.log(regexInsensitive.test("HELLO")); // true

// Character classes

// console.log(/[0-9]+/.test("123")); // true

// Quantifiers

```
// console.log(/a{2,4}/.test("aaa")); // true
```

```
// Groups / alternation
```

```
let petRegex = /(cat|dog)/;
```

```
// console.log(petRegex.exec("I have a dog"));
```

```
// Anchors
```

```
// console.log(/^hello$/.test("hello")); // true
```

```
// Lookahead
```

```
// console.log(/^d+(?=px)/.exec("20px")); // matches 20
```

```
// Lookbehind
```

```
// console.log(/(?<=\$)d+/.exec("$100")); // matches 100 if supported
```

```
// String methods with regex
```

```
// console.log("abc123".match(/^d+/));
```

```
// console.log("abc123".replace(/^d+/, "XYZ")); // abcXYZ
```

```
// console.log("a,b;c".split(/[,;]/)); // ['a','b','c']
```

```
/*
```

Regex limitation:

Complex regex is hard to maintain.

Best practice:

- keep regex simple
- explain tricky regex with comments

```
*/
```

```
// =====
```

```
// [21] JSON
```

```
// =====
```

```
/*
```

JSON = JavaScript Object Notation

Text format used for data exchange

\*/

```
let data = {
 name: "Dinesh",
 age: 30,
 skills: ["JS", "React"]
};
```

```
let jsonString = JSON.stringify(data);
// console.log(jsonString);
```

```
let parsed = JSON.parse(jsonString);
// console.log(parsed.name); // Dinesh
```

/\*

JSON limitations:

- no functions
- no undefined
- no Symbol
- Dates become strings
- circular references fail

\*/

```
let weird = {
 a: undefined,
 b: function () {},
 c: Symbol("x")
};
```

```
// console.log(JSON.stringify(weird)); // unsupported parts removed
```

```
// =====
```

```
// [22] ADVANCED BUILT-IN OBJECTS
```

```
// =====
```

```
// Map
```

```
/*
```

```
Map:
```

- key-value store
- keys can be any value
- preserves insertion order
- better than object for many add/remove operations or non-string keys

```
*/
```

```
const map = new Map();
```

```
map.set("name", "Dinesh");
```

```
map.set({ id: 1 }, "object key value");
```

```
// console.log(map.get("name")); // Dinesh
```

```
// console.log(map.size); // 2
```

```
// Set
```

```
/*
```

```
Set:
```

- stores unique values

```
*/
```

```
const set = new Set([1, 2, 2, 3]);
```

```
// console.log(set); // Set(3) {1,2,3}
```

```
set.add(4);
```

```
set.delete(2);
```

```
// console.log(set.has(3)); // true
```



```
// WeakMap / WeakSet
```

```
/*
```

WeakMap:

- keys must be objects
- weak references
- not iterable

Use case:

private metadata without preventing GC

```
*/
```

```
const wm = new WeakMap();
```

```
const objKey = {};
```

```
wm.set(objKey, "private data");
```

```
// Symbol
```

```
/*
```

Symbol:

- unique primitive identifier
- often used as collision-free object keys

```
*/
```

```
const uniqueId = Symbol("id");
```

```
const user = { [uniqueId]: 123 };
```

```
// console.log(user[uniqueId]); // 123
```

```
// Iterables and iterators
```

```
/*
```

Iterable protocol:

object has Symbol.iterator method

Iterator protocol:

object has next() method returning:

```
{ value, done }
```

```
*/
```

```
const myIterable = {
```

```
 data: [10, 20, 30],
```

```
 [Symbol.iterator]() {
```

```
 let index = 0;
```

```
 let arr = this.data;
```

```
 return {
```

```
 next() {
```

```
 if (index < arr.length) {
```

```
 return { value: arr[index++], done: false };
```

```
 }
```

```
 return { value: undefined, done: true };
```

```
 }
```

```
 };
```

```
 }
```

```
};
```

```
for (const value of myIterable) {
```

```
 // console.log(value);
```

```
}
```

```
// Generators
```

```
/*
```

Generators simplify iterator creation.

```
function* + yield
```

```
*/
```

```
function* idGenerator() {
```

```
 yield 1;
```

```
 yield 2;
```

```
yield 3;
}
```

```
const gen = idGenerator();
// console.log(gen.next()); // { value:1, done:false }
// console.log(gen.next()); // { value:2, done:false }
```

```
// Async generators
async function* asyncCounter() {
 yield 1;
 yield 2;
}
// for await (const val of asyncCounter()) { console.log(val); }
```

```
// Binary data
```

```
/*
```

ArrayBuffer:

raw binary buffer

Typed arrays:

views into buffer

DataView:

more flexible low-level access

```
*/
```

```
const buffer = new ArrayBuffer(8);
const view = new Uint8Array(buffer);
view[0] = 255;
// console.log(view[0]); // 255
```

```
// Proxy and Reflect
```

```
/*
```

Proxy lets you intercept operations like:

- get

- set
- deleteProperty
- has
- etc.

Reflect provides standard methods often used inside proxies

```
*/
```

```
const target = { count: 0 };
```

```
const proxy = new Proxy(target, {
 get(obj, prop, receiver) {
 return Reflect.get(obj, prop, receiver);
 },
 set(obj, prop, value, receiver) {
 if (prop === "count" && value < 0) {
 throw new RangeError("count cannot be negative");
 }
 return Reflect.set(obj, prop, value, receiver);
 }
});
```

```
proxy.count = 5;
```

```
// console.log(proxy.count); // 5
```

```
// Intl
```

```
const formatter = new Intl.NumberFormat("en-IN", {
 style: "currency",
 currency: "INR"
});
```

```
// console.log(formatter.format(123456.78)); // ₹1,23,456.78 approx
```

```
// =====
```

## // [23] DESTRUCTURING

// =====

/\*

Destructuring extracts values from arrays/objects.

Useful for:

- cleaner syntax
- default values
- parameter handling

\*/

```
const coords = [12.34, 56.78];
```

```
const [lat, lng] = coords;
```

```
// console.log(lat, lng);
```

```
const student = { name: "Raj", marks: 95, city: "Madurai" };
```

```
const { name: studentName, marks = 0, city = "Unknown" } = student;
```

```
// console.log(studentName, marks, city);
```

```
const nestedObj = { user: { profile: { title: "Developer" } } };
```

```
const { user: { profile: { title } } } = nestedObj;
```

```
// console.log(title); // Developer
```

```
const [head, ...tail] = [1, 2, 3, 4];
```

```
// console.log(head, tail); // 1 [2,3,4]
```

// =====

## // [24] SPREAD AND REST

// =====

/\*

Same syntax (...), different role depending on context:

Spread:

- expands iterable/object

Rest:

- collects remaining items

\*/

```
const numsA = [1, 2];
const numsB = [...numsA, 3, 4];
// console.log(numsB); // [1,2,3,4]
```

```
const mergedObj = { ...{ a: 1 }, ...{ b: 2 } };
// console.log(mergedObj); // {a:1,b:2}
```

```
function maxOfValues(...vals) {
 return Math.max(...vals);
}
// console.log(maxOfValues(1, 9, 3)); // 9
```

```
const { a: firstProp, ...remaining } = { a: 1, b: 2, c: 3 };
// console.log(firstProp, remaining); // 1 { b:2, c:3 }
```

```
// =====
```

```
// [25] MODULES
```

```
// =====
```

```
/*
```

Why modules?

- avoid global pollution
- improve organization
- improve reusability
- support code splitting and tree shaking

```
*/
```

/\*

## ES Modules (ESM)

math.js

-----  
export const PI = 3.14159;

export function add(a, b) {  
 return a + b;  
}

export default function multiply(a, b) {  
 return a \* b;  
}

-----  
app.js

import multiply, { PI, add as sum } from "./math.js";

console.log(PI);  
console.log(sum(2, 3));  
console.log(multiply(2, 3));

-----  
\*/

/\*

Named export:

- multiple per module

Default export:

- one main export per module

Dynamic import:

- load module lazily

```
*/
```

```
async function loadModuleExample() {
 // const mod = await import("./math.js");
}
```

```
// CommonJS
```

```
/*
```

Older Node style:

math.cjs

```
module.exports = {
 add: (a, b) => a + b
};
```

app.cjs

```
const { add } = require("./math.cjs");
console.log(add(2, 3));
*/
```

```
/*
```

ESM vs CommonJS:

- ESM is static and standard
- CommonJS is older Node module system
- ESM enables tree shaking

```
*/
```

```
// Circular dependency note
```

```
/*
```

If module A imports B and B imports A,



partially initialized values can appear.

Solution:

- refactor shared logic to third module
- simplify dependency graph

\*/

-----Part 5 -----

/\*

---

---

PART 5: ASYNC JAVASCRIPT, EVENT LOOP, DOM, EVENTS, FORMS, STORAGE, NETWORKING

---

---

\*/

// =====

// [26] ASYNCHRONOUS JAVASCRIPT

// =====

/\*

Synchronous code:

- one line waits for previous line

Asynchronous code:

- start an operation
- continue other work
- handle result later

Why async?

- network requests
- timers
- file I/O
- UI responsiveness

\*/

```
// Timers
```

```
setTimeout(() => {
 // console.log("Runs after delay");
}, 10);
```

```
const intervalId = setInterval(() => {
 // console.log("Interval tick");
 clearInterval(intervalId);
}, 10);
```

```
// Callback hell
```

```
/*
```

Example of bad nested style:

```
getUser(id, function(user) {
 getOrders(user.id, function(orders) {
 getPayment(orders[0], function(payment) {
 ...
 });
 });
});
```

Problem:

- deeply nested
- hard to read
- hard to maintain

Solution:

- Promises
- async/await

```
*/
```

```
// Promise basics
```

```
const promiseDemo = new Promise((resolve, reject) => {
 let success = true;

 if (success) {
 resolve("Operation successful");
 } else {
 reject(new Error("Operation failed"));
 }
});
```

```
promiseDemo
 .then(result => {
 // console.log(result);
 })
 .catch(error => {
 // console.error(error.message);
 })
 .finally(() => {
 // console.log("Promise completed");
 });
```

```
/*
```

Promise states:

- pending
- fulfilled
- rejected

```
*/
```

```
// Promise static methods
```

```
const p1 = Promise.resolve(1);
const p2 = Promise.resolve(2);
const p3 = Promise.reject(new Error("fail"));
```

```
Promise.all([p1, p2]).then(values => {
 // console.log(values); // [1, 2]
});
```

/\*

Promise.all:

- runs in parallel
- fails fast if any promise rejects
- use when all results required

\*/

```
Promise.allSettled([p1, p3]).then(results => {
 // console.log(results);
});
```

/\*

Promise.allSettled:

- waits for all
- gives status of each
- use when you want all outcomes regardless of failure

\*/

```
Promise.race([
 new Promise(res => setTimeout(() => res("fast"), 10)),
 new Promise(res => setTimeout(() => res("slow"), 50))
]).then(value => {
 // console.log(value); // fast
});
```

/\*

Promise.race:

- first settled wins

\*/

```
Promise.any([
 Promise.reject("A"),
 Promise.resolve("B")
]).then(value => {
 // console.log(value); // B
});
```

```
/*
Promise.any:
- first fulfilled wins
- ignores rejections until all fail
*/

// async / await
function fakeApi(value, delay) {
 return new Promise(resolve => setTimeout(() => resolve(value), delay));
}
```

```
async function asyncExample() {
 try {
 const a = await fakeApi("first", 10);
 const b = await fakeApi("second", 10);
 return [a, b];
 } catch (err) {
 throw err;
 }
}
```

```
// asyncExample().then(console.log); // ['first','second']
```

```
/*
Sequential async:
```

- await one, then next
  - use when second depends on first
- \*/

```
async function parallelExample() {
 const [a, b] = await Promise.all([
 fakeApi("first", 10),
 fakeApi("second", 10)
]);

 return [a, b];
}
```

```
// parallelExample().then(console.log);
```

/\*

Parallel async:

- use Promise.all when tasks are independent
- faster than sequential

\*/

```
// Cancellation pattern
```

```
const controller = new AbortController();
```

```
const signal = controller.signal;
```

```
// Example:
```

```
// fetch("/api/data", { signal });
```

```
// controller.abort();
```

```
// =====
```

```
// [27] EVENT LOOP AND SCHEDULING
```

```
// =====
```

/\*

Core pieces:

- Call stack
- Web APIs / Host APIs
- Callback queue / Task queue
- Microtask queue
- Event loop

\*/

/\*

Macrotasks:

- setTimeout
- setInterval
- I/O
- UI events

Microtasks:

- Promise.then/catch/finally
- queueMicrotask
- MutationObserver

\*/

/\*

Rule:

After current synchronous code finishes,  
microtasks run before next macrotask.

\*/

```
// console.log("start");
```

```
// setTimeout(() => console.log("timeout"), 0);
```

```
// Promise.resolve().then(() => console.log("promise"));
```

```
// queueMicrotask(() => console.log("microtask"));
```

```
// console.log("end");
```

```
/*
```

Expected order:

start

end

promise

microtask

timeout

```
*/
```

```
// =====
```

```
// [28] DOM AND BROWSER APIS
```

```
// =====
```

```
/*
```

DOM = Document Object Model

Represents HTML as a tree of objects

BOM = Browser Object Model

Includes:

- window

- history

- location

- navigator

```
*/
```

```
// Selecting elements
```

```
/*
```

```
document.getElementById()
```

```
document.getElementsByClassName()
```



```
document.getElementsByTagName()
```

```
document.querySelector()
```

```
document.querySelectorAll()
```

```
*/
```

```
/*
```

Browser example:

```
const el = document.getElementById("title");
```

```
console.log(el.textContent);
```

```
const btn = document.querySelector(".btn");
```

```
*/
```

```
// DOM manipulation
```

```
/*
```

textContent:

- safe text

- no HTML parsing

innerText:

- visible/rendered text

- may trigger layout

innerHTML:

- parses HTML

- dangerous with untrusted input (XSS risk)

```
*/
```

```
/*
```

Browser example:

```
const box = document.createElement("div");
```

```
box.textContent = "Hello DOM";
document.body.appendChild(box);
```

```
box.setAttribute("data-id", "101");
console.log(box.getAttribute("data-id"));
```

```
box.classList.add("active");
box.style.color = "red";
*/
```

```
// Traversal
```

```
/*
```

```
parentElement
```

```
children
```

```
firstElementChild
```

```
lastElementChild
```

```
nextElementSibling
```

```
previousElementSibling
```

```
*/
```

```
// =====
```

```
// [29] EVENTS
```

```
// =====
```

```
/*
```

Events let JS respond to actions:

- click
- input
- change
- submit
- keydown
- keyup
- mouseover

- mouseout
- focus
- blur
- load
- scroll

```
*/
```

```
/*
```

Basic example:

```
const btn = document.querySelector("#saveBtn");
btn.addEventListener("click", function(event) {
 console.log("Clicked");
 console.log(event.type); // click
});
*/
```

```
// removeEventListener
/*
```

To remove a listener, you must pass the same function reference.

```
*/
```

```
// Event flow
```

```
/*
```

Phases:

- 1) capturing
- 2) target
- 3) bubbling

```
*/
```

```
// preventDefault and stopPropagation
```

```
/*
```

preventDefault():

- stops default browser action

stopPropagation():

- stops event from moving further in chain

```
*/
```

```
/*
```

Form example:

```
form.addEventListener("submit", function(event) {
 event.preventDefault();
});
*/
```

```
// Event delegation
```

```
/*
```

Attach one listener to parent

Handle child clicks via event.target

Why use it?

- efficient
- works for dynamic children

```
*/
```

```
/*
```

Example:

```
const list = document.querySelector("#todoList");
list.addEventListener("click", function(event) {
 if (event.target.matches("li")) {
 console.log("Clicked:", event.target.textContent);
 }
});
*/
```

```
// =====
```

```
// [30] FORMS AND VALIDATION
```

```
// =====
```

```
/*
```

Form concepts:

- reading user input
- validating input
- submitting data
- preventing invalid submission

```
*/
```

```
/*
```

Constraint Validation API:

- required
- minlength
- maxlength
- pattern
- type="email"
- checkValidity()
- reportValidity()

```
*/
```

```
/*
```

Browser example:

```
const form = document.querySelector("#signupForm");
```

```
form.addEventListener("submit", (e) => {
```

```
 e.preventDefault();
```

```
 if (!form.checkValidity()) {
```

```
 form.reportValidity();
```

```
 return;
}

const fd = new FormData(form);
console.log(fd.get("email"));
});
*/

// FormData
/*
Use FormData to extract form values easily,
especially for file uploads or multipart forms.
*/

// =====
// [31] BROWSER STORAGE
// =====

/*
localStorage:
- persists until cleared

sessionStorage:
- lasts for tab session

cookies:
- small text storage sent with requests (depending on settings)

IndexedDB:
- structured async client-side database
*/

// localStorage example
```

```
/*
```

```
localStorage.setItem("theme", "dark");
```

```
console.log(localStorage.getItem("theme")); // dark
```

```
localStorage.removeItem("theme");
```

```
*/
```

```
// Object storage
```

```
/*
```

```
const settings = { mode: "dark" };
```

```
localStorage.setItem("settings", JSON.stringify(settings));
```

```
const saved = JSON.parse(localStorage.getItem("settings"));
```

```
console.log(saved.mode);
```

```
*/
```

```
/*
```

Limitations:

- localStorage/sessionStorage store strings only
- cookies are small and may increase network overhead
- IndexedDB is powerful but more complex

```
*/
```

```
// =====
```

```
// [32] NETWORKING APIS
```

```
// =====
```

```
/*
```

Fetch API:

modern API for HTTP requests

returns promises

```
*/
```

```
/*
```

```
fetch("/api/users")

 .then(response => {
 if (!response.ok) throw new Error("HTTP error");
 return response.json();
 })

 .then(data => console.log(data))
 .catch(err => console.error(err));
*/
```

```
/*
```

Async/await version:

```
async function loadUsers() {
 const response = await fetch("/api/users");
 if (!response.ok) throw new Error("Request failed");
 return response.json();
}
*/
```

```
/*
```

XMLHttpRequest:

older API, still appears in legacy code

AJAX:

concept of asynchronous requests without full page reload

```
*/
```

```
// CORS
```

```
/*
```

Cross-Origin Resource Sharing

Browser security mechanism controlling cross-origin requests

```
*/
```



// WebSockets

/\*

Persistent full-duplex connection

Best for:

- chat apps
- live dashboards
- real-time games

\*/

// SSE (Server-Sent Events)

/\*

Server pushes messages to browser

One-way: server -> client

Useful for:

- notifications
- live feeds

\*/

// =====

// [33] ADVANCED BROWSER APIS

// =====

/\*

Geolocation API

- get current location with permission

Canvas API

- draw graphics

Web Workers

- run script in background thread
- cannot directly access DOM

## Service Workers

- caching
- offline support
- PWA features

## Notifications API

- system notifications with permission

## Clipboard API

- programmatic copy/paste

## Intersection Observer

- detect when element enters viewport
- useful for lazy loading

## Mutation Observer

- watch DOM changes

## Resize Observer

- watch element resizing

## History API

- SPA navigation with pushState/popstate

## Location API

- inspect URL

\*/

/\*

## Examples of use cases:

- IntersectionObserver -> infinite scroll / lazy-loading
- Web Worker -> CPU-heavy parsing without UI freeze
- Service Worker -> offline web app

- ResizeObserver -> adaptive layouts

\*/

-----Part 6-----

/\*

=====

PART 6: NODE.JS, MEMORY/PERFORMANCE, FUNCTIONAL PROGRAMMING, INTERVIEW  
RECALL

=====

\*/

// =====

// [34] NODE.JS CONCEPTS

// =====

/\*

Node.js:

JavaScript runtime outside the browser

Built on V8 + Node APIs

\*/

/\*

Why Node.js?

- use JavaScript on server

- build APIs

- scripts/automation

- tooling

- real-time apps

\*/

// Global objects in Node

/\*

- global

- process
- Buffer
- setTimeout
- setInterval

\*/

// process

/\*

process provides:

- environment variables
- current working dir
- argv command-line args
- exit control

\*/

// Node example

/\*

console.log(process.env.NODE\_ENV);

console.log(process.cwd());

console.log(process.argv);

\*/

// Buffer

/\*

Buffer handles raw binary data

Useful for:

- file data
- streams
- network packets

\*/

// fs module

/\*

File system operations:

- read file
- write file
- create directories
- etc.

\*/

/\*

Node example:

```
const fs = require("fs");
fs.writeFileSync("demo.txt", "Hello Node");
console.log(fs.readFileSync("demo.txt", "utf8"));
*/
```

// path module

/\*

Path utilities:

- path.join
- path.resolve
- path.basename

\*/

// http module

/\*

Can create raw HTTP servers

Example:

```
const http = require("http");

const server = http.createServer((req, res) => {
 res.end("Hello");
});

server.listen(3000);
```

```
*/
```

```
// streams
```

```
/*
```

Streams process data in chunks instead of loading all at once.

Useful for:

- large files
- memory efficiency
- network transfer

```
*/
```

```
// EventEmitter
```

```
/*
```

Node uses event-driven pattern heavily.

```
*/
```

```
/*
```

```
const EventEmitter = require("events");
```

```
const emitter = new EventEmitter();
```

```
emitter.on("greet", (name) => {
```

```
 console.log("Hello", name);
```

```
});
```

```
emitter.emit("greet", "Dinesh");
```

```
*/
```

```
// package.json
```

```
/*
```

Contains:

- project metadata
- dependencies
- scripts

- module type

\*/

// npm / yarn / pnpm

/\*

Package managers for installing dependencies and running scripts

\*/

// environment variables

/\*

Use process.env for configuration.

Never hardcode secrets in source code.

\*/

// REPL

/\*

Node REPL = interactive shell for testing JS quickly

\*/

// child\_process

/\*

Run external commands/processes

Use cases:

- automation

- build tools

- system integration

\*/

// worker\_threads

/\*

Use real threads in Node for CPU-heavy work

Different from normal event-loop async I/O

\*/

```
// CommonJS vs ESM in Node
```

```
/*
```

CommonJS:

- require
- module.exports

ESM:

- import/export
- newer standard

Use ESM in modern apps when possible,  
but understand both for interviews and legacy projects.

```
*/
```

```
// =====
```

```
// [35] MEMORY AND PERFORMANCE
```

```
// =====
```

```
/*
```

Stack vs Heap

Stack:

- call frames
- fast
- structured storage

Heap:

- objects / dynamic allocation
- managed by garbage collector

```
*/
```

```
function getFirst(arr) {
```



```
 return arr[0]; // O(1)
}
```

```
function findValue(arr, target) {
 for (let i = 0; i < arr.length; i++) {
 if (arr[i] === target) return i;
 }
 return -1; // O(n)
}
```

```
function bubbleSortLike(arr) {
 for (let i = 0; i < arr.length; i++) {
 for (let j = 0; j < arr.length - 1; j++) {
 if (arr[j] > arr[j + 1]) {
 [arr[j], arr[j + 1]] = [arr[j + 1], arr[j]];
 }
 }
 }
 return arr; // O(n^2)
}
```

```
/*
```

Primitive vs reference storage:

- primitives copied by value
- objects referenced

```
*/
```

```
// Garbage collection
```

```
/*
```

JavaScript automatically frees unreachable memory.

Common strategy:

- mark and sweep

```
*/
```

// Memory leaks

/\*

Common causes:

- forgotten intervals/timeouts
- event listeners not removed
- accidental globals
- closures holding large objects unnecessarily
- detached DOM nodes

\*/

// Memory leak prevention

```
function startInterval() {
 const id = setInterval(() => {
 // repeating task
 }, 1000);

 return () => clearInterval(id);
}
```

```
const stop = startInterval();
stop();
```

/\*

Big-O basics:

- $O(1)$  constant
- $O(\log n)$  logarithmic
- $O(n)$  linear
- $O(n \log n)$  efficient sort class
- $O(n^2)$  nested loops

\*/

// Lazy loading

/\*

Load resources only when needed

Examples:

- images
- routes/components
- modules

\*/

// Code splitting

/\*

Split app into smaller bundles

Load on demand

Improves initial performance

\*/

// Memoization

/\*

Cache expensive function results

Best for:

- same inputs repeated
- deterministic functions

\*/

// Debounce vs Throttle

/\*

Debounce:

- wait until activity stops

Example: search input

Throttle:

- allow execution at fixed intervals

Example: scroll event

\*/

```
// Reflow vs repaint
```

```
/*
```

Reflow:

- layout recalculation

Repaint:

- visual redraw

Why important?

Excessive reflow hurts performance.

```
*/
```

```
// =====
```

```
// [36] FUNCTIONAL PROGRAMMING IN JAVASCRIPT
```

```
// =====
```

```
/*
```

Core FP concepts:

- first-class functions
- higher-order functions
- pure functions
- immutability
- referential transparency
- composition

```
*/
```

```
const addTax = price => price * 1.18;
```

```
const formatPrice = price => `₹${price.toFixed(2)}`;
```

```
const compose = (f, g) => x => f(g(x));
```

```
const formatFinalPrice = compose(formatPrice, addTax);
```

```
// console.log(formatFinalPrice(100)); // ₹118.00
```

```
// Immutability
```

```
const state = {
 count: 0,
 user: { name: "A" }
};
```

```
const nextState = {
 ...state,
 count: state.count + 1,
 user: { ...state.user, name: "B" }
};
```

```
// console.log(state);
```

```
// console.log(nextState);
```

```
/*
```

Pure functions:

- predictable
- easier to test
- easier to reason about

```
*/
```

```
function pureDouble(x) {
 return x * 2;
}
```

```
/*
```

Referential transparency:

pureDouble(4) can always be replaced by 8

without changing program meaning

```
*/
```

// =====

## // [37] PRACTICAL INTERVIEW SCENARIOS

// =====

/\*

1) Why prefer let/const over var?

Answer:

- block scoping
- fewer bugs
- clearer intent
- avoids var hoisting confusion

2) Difference between == and ===?

Answer:

- == does coercion
- === compares type + value
- prefer ===

3) Why is 0.1 + 0.2 not exactly 0.3?

Answer:

- binary floating-point precision issue

4) What is closure and real use case?

Answer:

- inner function remembers lexical variables
- use for private state, memoization, callbacks

5) Difference between for...in and for...of?

Answer:

- for...in -> keys/properties
- for...of -> iterable values

6) What is event delegation?

Answer:

- parent listener handles child events using bubbling
- useful for dynamic lists and performance

7) What is prototype chain?

Answer:

- property lookup chain through linked prototypes

8) Difference between synchronous and asynchronous code?

Answer:

- sync blocks
- async lets other work continue while waiting

9) Promise vs async/await?

Answer:

- async/await is cleaner syntax built on promises

10) When use Map over Object?

Answer:

- keys of any type
- predictable iteration
- frequent insert/delete

\*/

// =====

// [38] COMMON PITFALLS CHECKLIST

// =====

/\*

- typeof null === "object"
- NaN !== NaN
- [] == false is true (coercion weirdness)
- {} + [] and [] + {} can surprise

- sort() is lexical by default
- return followed by newline causes ASI problem
- arrow functions do not have own this
- shallow copy doesn't clone nested objects
- localStorage stores strings only
- Promise microtasks run before setTimeout macrotasks
- deleting array element leaves hole
- class syntax still uses prototypes internally

\*/

// =====

// [39] MINI PRACTICE QUESTIONS

// =====

/\*

Q1:

console.log(typeof NaN);

A: "number"

Q2:

console.log([] == false);

A: true

Q3:

console.log("5" - 2, "5" + 2);

A: 3 and "52"

Q4:

Explain closure with code:

```
function outer() {
```

```
 let x = 10;
```

```
 return function inner() {
```

```
 return x;
```



```
};
}
```

Q5:

Difference between shallow copy and deep copy?

Q6:

When use Promise.all vs Promise.allSettled?

Q7:

Why avoid innerHTML with untrusted input?

A: XSS risk

Q8:

Why debounce a search box?

A: reduce unnecessary API calls

```
*/
```

```
// =====
```

```
// [40] FINAL RECALL MAP
```

```
// =====
```

```
/*
```

Ultra-short recall summary:

Basics:

- JS is dynamic, single-threaded, prototype-based

Variables:

- prefer const, then let
- understand scope, hoisting, TDZ

Types:

- 7 primitives + object
- know coercion and equality traps

#### Functions:

- first-class
- closures
- this
- callbacks
- HOFs

#### Objects:

- properties
- methods
- descriptors
- prototypes
- classes

#### Async:

- callbacks -> promises -> async/await
- event loop -> microtasks/macrotasks

#### Browser:

- DOM
- events
- forms
- storage
- networking
- APIs

#### Node:

- process
- fs
- path

- Buffer
- streams
- modules

#### Performance:

- memory
- GC
- leaks
- Big-O
- debounce/throttle

#### FP:

- pure functions
- immutability
- composition

\*/

// End of full revision guide

